Integração do Spring Actuator com Prometheus

A integração no desenvolvimento de software refere-se ao processo de combinar diferentes componentes, serviços ou sistemas para que trabalhem juntos de maneira coesa. Esse conceito é fundamental em ambientes distribuídos, onde várias aplicações, sistemas legados ou serviços em nuvem precisam interagir para entregar funcionalidades completas.
Existem diversos tipos de integrações, dentre elas podemos citar:

1. Integração de Sistema

A integração de sistemas conecta diferentes sistemas de software, que podem ser criados em diferentes linguagens e com diferentes arquiteturas, para que possam se comunicar. Por exemplo, uma aplicação moderna pode integrar-se a um sistema legado para obter dados históricos.

2. Integração de Aplicações

Envolve conectar diferentes aplicativos de uma organização para compartilhar dados ou funcionalidades, por exemplo, integrar um sistema de CRM com uma plataforma de automação de marketing.

3. Integração de Serviços

Com o surgimento de arquiteturas baseadas em microsserviços, pequenos serviços autônomos são integrados para fornecer uma solução unificada.

4. Integração de Dados

É uma integração focada na sincronização de dados entre sistemas. Um exemplo é a integração entre um banco de dados relacional e um sistema de análise em tempo real, onde os dados são replicados e mantidos sincronizados entre diferentes fontes.

5. Integração Contínua

Envolve a prática de integrar continuamente o código produzido pelos desenvolvedores em um repositório centralizado, essa integração contínua é feita com o uso de pipelines de CI/CD, permitindo testes e validação automáticos.

Métricas

A personalização de métricas é uma maneira eficaz de criar, coletar e monitorar métricas específicas da sua aplicação, além das métricas padrão oferecidas pelo Spring Actuator. Isso é útil quando você deseja medir aspectos específicos do seu sistema, como o tempo de execução de operações personalizadas, o número de acessos a determinado serviço, ou o sucesso de operações críticas.
Para que a visualização de métricas seja feita de maneira eficaz, podemos citar alguns fatores importantes como:

1. Compreensão rápida de dados complexos

· É importante, pois traduz grandes volumes de dados em representações visuais de fácil entendimento como gráficos e tabelas, facilitando a interpretação de tendências e padrões, acelerando o diagnóstico e a resposta a problemas.

2. Monitoramento em tempo real

· É importante, pois permite que problemas sejam identificados e tratados assim que surgem, minimizando o impacto de falhas nos sistemas. Isso melhora a disponibilidade dos serviços, reduzindo tempo de inatividade e garantindo a continuidade das operações.

3. Identificação de Tendências e Padrões

· Análise de métricas ao longo do tempo ajuda a entender padrões de uso e comportamento, permitindo que as equipes antecipem problemas e otimizem a infraestrutura, suportando a previsão de demandas e planejamento de recursos, evitando a sobrecarga de sistemas.

4. Melhoria na Tomada de Decisão

· A visualização facilita decisões baseadas em dados concretos, alinhando estratégias técnicas e de negócios, aumentando a eficácia das decisões e estratégias, assegurando que as ações estejam sempre embasadas em uma visão clara da realidade dos sistemas e dos negócios.

GitHub

Confira o repositório da disciplina no GitHub: https://github.com/FaculdadeDescomplica/Monitoramento.

Referência Bibliográfica

Brazil, Brian. Prometheus Up & Running. Sebastopol, California: O'Reilly Media, 2015.

Prometheus. Documentação do Spring. Disponível em https://docs.spring.io/spring-boot/api/rest/actuator/prometheus.html. Acesso em 12 de novembro de 2024.

Sánchez, José Carlos Valero. Spring Boot Actuator. Disponível em https://www.baeldung.com/spring-boot-actuators. Acesso em 11 de novembro de 2024.

Building a RESTful Web Service with Spring Boot Actuator. Documentação do Spring. Disponível em https://spring.io/guides/gs/actuator-service. Acesso em 11 de novembro de 2024.

Ir para exercício